Machine Learning Applications

Pandas — Data Analysis with Python

NUS Institute of Systems Science · Postgraduate Programme

Why Pandas?

Pandas is Python's primary library for **data manipulation and analysis**. Built on top of NumPy, it provides labelled data structures — **Series** (1D) and **DataFrame** (2D) — that make working with structured/tabular data intuitive and fast.

NumPy vs Pandas

Feature	NumPy	Pandas
Structure	Homogeneous arrays	Labelled tables (mixed types)
Indexing	Integer position only	Labels + positions (loc/iloc)
Missing data	No built-in support	NaN handling built-in
I/O	Limited	CSV, Excel, SQL, JSON…
Use case	Numerical computation	Data wrangling & EDA

💡 In ML workflows, Pandas is used for the **first 80%** of the work: loading data, exploring, cleaning, and feature engineering. The model training part is only the last 20%.

why_pandas.py ▶ Run

```
import pandas as pd
import numpy as np

# Create a DataFrame from a dictionary
df = pd.DataFrame({
    "name":   ["Alice", "Bob", "Charlie"],
    "age":    [25, 30, 35],
    "salary": [50000, 60000, 70000]
})
print(df)
print(f"\nType: {type(df)}")
print(f"Shape: {df.shape}")
print(f"Columns: {list(df.columns)}")
print(f"Dtypes:\n{df.dtypes}")
```

OUTPUT

Part 1

Series — Labelled 1D Arrays

Pandas Series

A **Series** is a 1D labelled array. Think of it as a single column in a spreadsheet. Each element has an **index label** (default: 0, 1, 2… or custom names). Values are **mutable**, but the size is **immutable** after creation.

Key Properties

- Created with `pd.Series(data, index=...)`
- Supports custom index labels (strings, dates, etc.)
- `.iloc[n]` — access by position (integer)
- `.loc['name']` — access by label (name)
- **iloc slicing** is exclusive: `[1:3]` → items 1, 2
- **loc slicing** is inclusive: `['a':'c']` → items a, b, c

💡 The `iloc` vs `loc` distinction is one of the most common sources of confusion and bugs. Remember: **i**loc = **i**nteger position, **l**oc = **l**abel.

series.py ▶ Run

```
import pandas as pd

# Default integer index
s1 = pd.Series([21, 32, 45])
print(f"Default index:\n{s1}\n")

# Custom named index
s2 = pd.Series([21, 32, 45],
               index=['mary', 'john', 'james'])
print(f"Named index:\n{s2}\n")

# Access by position (iloc) vs label (loc)
print(f"iloc[1] = {s2.iloc[1]}")
print(f"loc['john'] = {s2.loc['john']}")

# CRITICAL: loc slicing is INCLUSIVE!
print(f"\niloc[0:2]:\n{s2.iloc[0:2]}")  # mary, john
```

OUTPUT

Part 2

DataFrame — The Core Data Structure for ML

Creating DataFrames

A **DataFrame** is a 2D labelled table with **column names** and **row index**. It's both value-mutable and size-mutable (you can add/remove columns). You can create DataFrames from dicts, lists of lists, NumPy arrays, or by reading files.

Creation Methods

- `pd.DataFrame(dict)` — from dictionary (most common)
- `pd.DataFrame(list_of_lists, columns=...)`
- `pd.read_csv('file.csv')` — from CSV file
- `pd.read_excel('file.xlsx')` — from Excel
- `pd.DataFrame(np_array, columns=...)`

💡 The **dict-based** creation is the most intuitive: each key becomes a column name, each value is a list of that column's data. This maps naturally to how we think about tabular data.

create_df.py ▶ Run

```
import pandas as pd

# Method 1: From dictionary (most common!)
df = pd.DataFrame({
    'name': ['Mary', 'John', 'James'],
    'age': [21, 32, 45],
    'height': [1.7, 1.8, 1.6],
    'weight': [55, 65, 50]
})
print(f"From dict:\n{df}\n")

# Method 2: From list of lists + column names
df2 = pd.DataFrame(
    [[21, 1.7, 55], [32, 1.8, 65], [45, 1.6, 50]],
    columns=['age', 'height', 'weight'],
    index=['mary', 'john', 'james']
)
```

OUTPUT

Exploring Your Data

The first thing you do with any dataset: **explore it**. Pandas provides quick inspection methods — `head()`, `tail()`, `info()`, `describe()`, `shape`, `dtypes` — to understand your data before any analysis.

Essential Inspection

- `df.head(n)` — first n rows (default 5)
- `df.tail(n)` — last n rows
- `df.shape` — (rows, columns) tuple
- `df.info()` — column types, non-null counts, memory
- `df.describe()` — statistics (mean, std, min, max, quartiles)
- `df.dtypes` — data type per column
- `df.value_counts()` — frequency of unique values

💡 `df.describe()` and `df.info()` are the two most important commands when you first load a dataset. They reveal data types, missing values, and statistical distributions instantly.

explore.py ▶ Run

```
import pandas as pd
import numpy as np

# Create a sample dataset
np.random.seed(42)
df = pd.DataFrame({
    'name': ['Alice','Bob','Charlie','Diana','Eve',
             'Frank','Grace','Hank','Iris','Jack'],
    'age': np.random.randint(20, 60, 10),
    'salary': np.random.randint(30000, 90000, 10),
    'dept': np.random.choice(['Sales','Eng','HR'], 10)
})
print(f"Shape: {df.shape}\n")
print(f"First 3 rows:\n{df.head(3)}\n")
print(f"Statistics:\n{df.describe()}\n")
print(f"Dept counts:\n{df['dept'].value_counts()}")
```

OUTPUT

Part 3

Selecting, Filtering & Indexing Data

Selecting Columns & Rows

Select columns with `df['col']` (returns Series) or `df[['col1','col2']]` (returns DataFrame). Select rows with `df.iloc[pos]` (by position) or `df.loc[label]` (by label). This is the foundation of all data manipulation.

Selection Patterns

- `df['age']` — one column → Series
- `df[['age','name']]` — multiple columns → DataFrame
- `df.iloc[0]` — first row → Series
- `df.iloc[[0]]` — first row → DataFrame
- `df.iloc[1:3, 0:2]` — rows 1-2, cols 0-1
- `df.loc['john', 'age']` — specific cell

🧠 Quick Think

What's the difference between `df.iloc[1]` and `df.iloc[[1]]`?

- They return the same thing
- iloc[1] → Series, iloc[[1]] → DataFrame
- iloc[[1]] raises an error

selection.py ▶ Run

```
import pandas as pd

df = pd.DataFrame({
    'age': [21, 32, 45],
    'height': [1.7, 1.8, 1.6],
    'weight': [55, 65, 50]
}, index=['mary', 'john', 'james'])
print(f"DataFrame:\n{df}\n")

# Column selection
print(f"df['age'] (Series):\n{df['age']}\n")
print(f"df[['age','weight']] (DataFrame):")
print(f"{df[['age','weight']]}\n")

# Row by position (iloc)
print(f"iloc[1] (Series):\n{df.iloc[1]}\n")
print(f"iloc[[1]] (DataFrame):\n{df.iloc[[1]]}\n")
```

OUTPUT

Boolean Filtering

Filter rows by condition: `df[df['col'] > value]`. This works like NumPy boolean indexing — the condition creates a boolean mask, and only True rows are kept. Combine conditions with & (and), | (or).

Filtering Patterns

- `df[df['age'] > 30]` — rows where age > 30
- `df[(df['age'] > 20) & (df['age'] < 40)]` — combined
- `df[df['name'].isin(['Alice','Bob'])]` — match list
- `df[df['name'].str.contains('ar')]` — string match
- `df.query('age > 30 and salary > 50000')` — query syntax

💡 Parentheses are **mandatory** when combining conditions with & and |. Without them, Python's operator precedence causes errors. This is the #1 filtering mistake.

filtering.py ▶ Run

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice','Bob','Charlie','Diana','Eve'],
    'age': [25, 30, 35, 28, 42],
    'dept': ['Eng','Sales','Eng','HR','Sales'],
    'salary': [70000, 55000, 80000, 48000, 62000]
})
print(f"All data:\n{df}\n")

# Simple filter
print(f"Age > 30:\n{df[df['age'] > 30]}\n")

# Combined (MUST use parentheses!)
mask = (df['age'] > 25) & (df['dept'] == 'Eng')
print(f"Age>25 AND Eng:\n{df[mask]}\n")
```

OUTPUT

Part 4

Data Wrangling — Modifying, Cleaning & Transforming

Adding & Modifying Columns

DataFrames are **size-mutable** — you can add new columns, modify existing ones, rename them, or drop them. New columns are typically computed from existing ones (feature engineering).

Column Operations

- `df['new_col'] = values` — add/overwrite column
- `df['bmi'] = df['weight'] / df['height']**2` — computed
- `df.rename(columns={'old':'new'})` — rename
- `df.drop(columns=['col'])` — remove column
- `df.drop(index=['row'])` — remove row
- `df.apply(func)` — apply function to each column/row

💡 In ML, **feature engineering** means creating new columns from existing data — computing ratios, extracting date parts, binning values, etc. This is where domain knowledge meets data science.

modify_cols.py ▶ Run

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Mary', 'John', 'James'],
    'height': [1.7, 1.8, 1.6],
    'weight': [55, 65, 50]
})
print(f"Original:\n{df}\n")

# Add computed column (feature engineering!)
df['bmi'] = df['weight'] / df['height']**2
print(f"With BMI:\n{df}\n")

# Categorise with apply
df['status'] = df['bmi'].apply(
    lambda x: 'Normal' if x < 25 else 'Overweight'
)
```

OUTPUT

Handling Missing Data

Real-world data is messy — **missing values** (NaN) are everywhere. Pandas uses NaN (Not a Number) to represent missing data. You must decide: **drop** the rows/columns, or **fill** (impute) with a sensible value.

Missing Data Methods

- `df.isnull()` — boolean mask of NaN locations
- `df.isnull().sum()` — count NaNs per column
- `df.dropna()` — drop rows with any NaN
- `df.dropna(subset=['col'])` — drop if col is NaN
- `df.fillna(value)` — fill all NaNs with a value
- `df['col'].fillna(df['col'].mean())` — impute with mean

💡 Choosing between dropping and filling is a critical ML decision. Dropping loses data; filling introduces assumptions. For small datasets, **imputing with the median** is often safer than the mean (less sensitive to outliers).

missing_data.py

▶ Run

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'age': [25, np.nan, 35, 28],
    'salary': [50000, 60000, np.nan, 48000]
})
print(f"With missing values:\n{df}\n")

# Detect missing values
print(f"NaN counts:\n{df.isnull().sum()}\n")

# Option 1: Drop rows with any NaN
print(f"After dropna():\n{df.dropna()}\n")

# Option 2: Fill with column mean (impute)
```

OUTPUT

Sorting & Ranking

Sort DataFrames by column values with `sort_values()` or by index with `sort_index()`. Sorting is essential for ranking, finding top-N items, and understanding data distributions.

Sorting Methods

- `df.sort_values('col')` — ascending by default
- `df.sort_values('col', ascending=False)` — descending
- `df.sort_values(['col1','col2'])` — multi-column sort
- `df.nlargest(n, 'col')` — top n rows
- `df.nsmallest(n, 'col')` — bottom n rows
- `df.reset_index(drop=True)` — reset to 0,1,2…

💡 `nlargest()` and `nsmallest()` are faster than sorting the entire DataFrame when you only need the top/bottom few rows — useful for large datasets.

sorting.py ▶ Run

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice','Bob','Charlie','Diana','Eve'],
    'age': [25, 30, 35, 28, 42],
    'salary': [70000, 55000, 80000, 48000, 62000]
})

# Sort by salary (descending)
print(f"By salary (desc):")
print(df.sort_values('salary', ascending=False))

# Top 3 earners
print(f"\nTop 3 earners:")
print(df.nlargest(3, 'salary'))

# Sort by multiple columns
```

OUTPUT

Part 5

GroupBy, Aggregation & Merging

GroupBy — Split-Apply-Combine

`groupby()` splits data into groups by a column, applies an aggregation (sum, mean, count…), and combines results. This is the Pandas equivalent of SQL's `GROUP BY` — one of the most powerful operations for data analysis.

GroupBy Pattern

- `df.groupby('col').mean()` — mean per group
- `df.groupby('col').agg(['mean','std'])` — multiple stats
- `df.groupby('col')['target'].sum()` — specific column
- `df.groupby(['col1','col2']).count()` — multi-group

💡 GroupBy is how you answer questions like "What is the average salary per department?" or "Which category has the highest count?" — the bread and butter of exploratory data analysis (EDA).

groupby.py ▶ Run

```
import pandas as pd

df = pd.DataFrame({
    'dept': ['Eng','Sales','Eng','HR','Sales','HR'],
    'name': ['Alice','Bob','Charlie','Diana','Eve','Frank'],
    'salary': [70000, 55000, 80000, 48000, 62000, 52000]
})
print(f"Data:\n{df}\n")

# Average salary per department
print(f"Mean salary by dept:")
print(df.groupby('dept')['salary'].mean())

# Multiple aggregations
print(f"\nMultiple stats:")
print(df.groupby('dept')['salary'].agg(
    ['mean', 'min', 'max', 'count']
```

OUTPUT

Merging & Concatenating

`pd.merge()` joins two DataFrames on a common column (like SQL JOIN). `pd.concat()` stacks DataFrames vertically (adding rows) or horizontally (adding columns).

Join Types

- `pd.merge(df1, df2, on='key')` — inner join (default)
- `how='left'` — keep all rows from left
- `how='right'` — keep all rows from right
- `how='outer'` — keep all rows from both
- `pd.concat([df1, df2])` — stack vertically
- `pd.concat([df1, df2], axis=1)` — side by side

💡 In ML, you frequently merge datasets from different sources — joining feature tables with label tables, or combining train/test metadata. Understanding join types prevents data leakage and missing rows.

merge.py ▶ Run

```
import pandas as pd

# Two related tables
employees = pd.DataFrame({
    'emp_id': [1, 2, 3, 4],
    'name': ['Alice', 'Bob', 'Charlie', 'Diana'],
    'dept_id': [10, 20, 10, 30]
})

departments = pd.DataFrame({
    'dept_id': [10, 20, 40],
    'dept_name': ['Engineering', 'Sales', 'Marketing']
})

# Inner join (default) — only matching rows
inner = pd.merge(employees, departments, on='dept_id')
print(f"Inner join:\n{inner}\n")
```

OUTPUT

Apply — Custom Transformations

`apply()` runs a function on each element, row, or column. Combined with `lambda`, it's the Swiss Army knife for data transformation — anything you can express as a function, you can apply to a DataFrame.

Apply Patterns

- `df['col'].apply(func)` — apply to each element
- `df.apply(func, axis=0)` — apply to each column
- `df.apply(func, axis=1)` — apply to each row
- `df['col'].map({'A':1, 'B':2})` — value mapping
- `df.applymap(func)` — apply to every cell

💡 When vectorised operations aren't possible (complex conditional logic, string parsing), `apply()` is your fallback. It's slower than vectorisation but much faster and cleaner than Python for-loops.

apply.py

▶ Run

```
import pandas as pd

df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'score': [85, 62, 91],
    'grade_letter': ['A', 'B', 'A']
})
print(f"Original:\n{df}\n")

# Apply lambda to transform a column
df['pass_fail'] = df['score'].apply(
    lambda x: 'Pass' if x >= 70 else 'Fail'
)

# Map categorical values to numbers
df['grade_num'] = df['grade_letter'].map(
    {'A': 4, 'B': 3, 'C': 2}
```

OUTPUT

Summary & Next Steps

Pandas is how you prepare data for ML. Master these operations and you'll spend less time fighting data and more time building models. Next: **Linear & Logistic Regression**.

What We Covered

- Series: labelled 1D arrays, iloc vs loc
- DataFrame creation: from dict, lists, files
- Exploration: head, describe, info, dtypes
- Selection: columns, rows, iloc/loc, slicing
- Boolean filtering with conditions
- Adding/modifying columns (feature engineering)
- Handling missing data: dropna, fillna
- Sorting and ranking
- GroupBy — split-apply-combine
- Merging and concatenation
- Apply for custom transformations

Practice Resources

- Kaggle — Learn Pandas (free, interactive)
- Complete Pandas Tutorial for Beginners
- Official Pandas Tutorials

💡 **Key practice areas:** boolean filtering, groupby aggregation, and handling missing data. These three skills cover 80% of real-world data wrangling tasks in ML projects.

Thank You

Questions & Discussion

Machine Learning Applications · NUS Institute of Systems Science